Technische Universität Dresden

Dr. Thomas Springer
Faculty of Computer Science | Chair of Distributed and Networked Systems

Distributed Systems
01_Foundations

winter term 2026/2027

Is it ok to record the lectures?
- audio and slides only

Introduction



Course Organisation

All information and material available in OPAL

- https://bildungsportal.sachsen.de/opal/auth/RepositoryEntry/50905251844

- ➢ Please register, if not done yet
- ➢ Subscribe to Announcements

Written Exam

- Exam format for the course is determined in the module description
- Exam date is centrally set and will be announced as early as possible

Please plan accordingly, since no exceptions regarding either the exam format or the exam date can be made.

Course Schedule

22-Oct-26	Foundations	
29-Oct-26	*no lecture and seminar*	
5-Nov-26	Distributed System Architectures	
12-Nov-26	Communication	
19-Nov-26	Distributed Transactions	Foundations
26-Nov-26	Security Services	Distributed System Architectures
3-Dec-26	Resource Discovery	Communication
10-Dec-26	SOA and Business Processes	Distributed Transactions
17-Dec-26	Cloud Computing	Security Services
24-Dec-26	Christmas Holydays	
31-Dec-26	Christmas Holydays	
7-Jan-27	Processing Big Data	Resource Discovery
14-Jan-27	Engineering Distributed Systems	SOA and Business Processes
21-Jan-27	*buffer*	Cloud Computing
28-Jan-27	Processing Big Data, Engineering + Exam prep.	
4-Feb-27	*no lecture and seminar*	

lecture
seminar
homework
holydays

Lerning Objectives of the Course

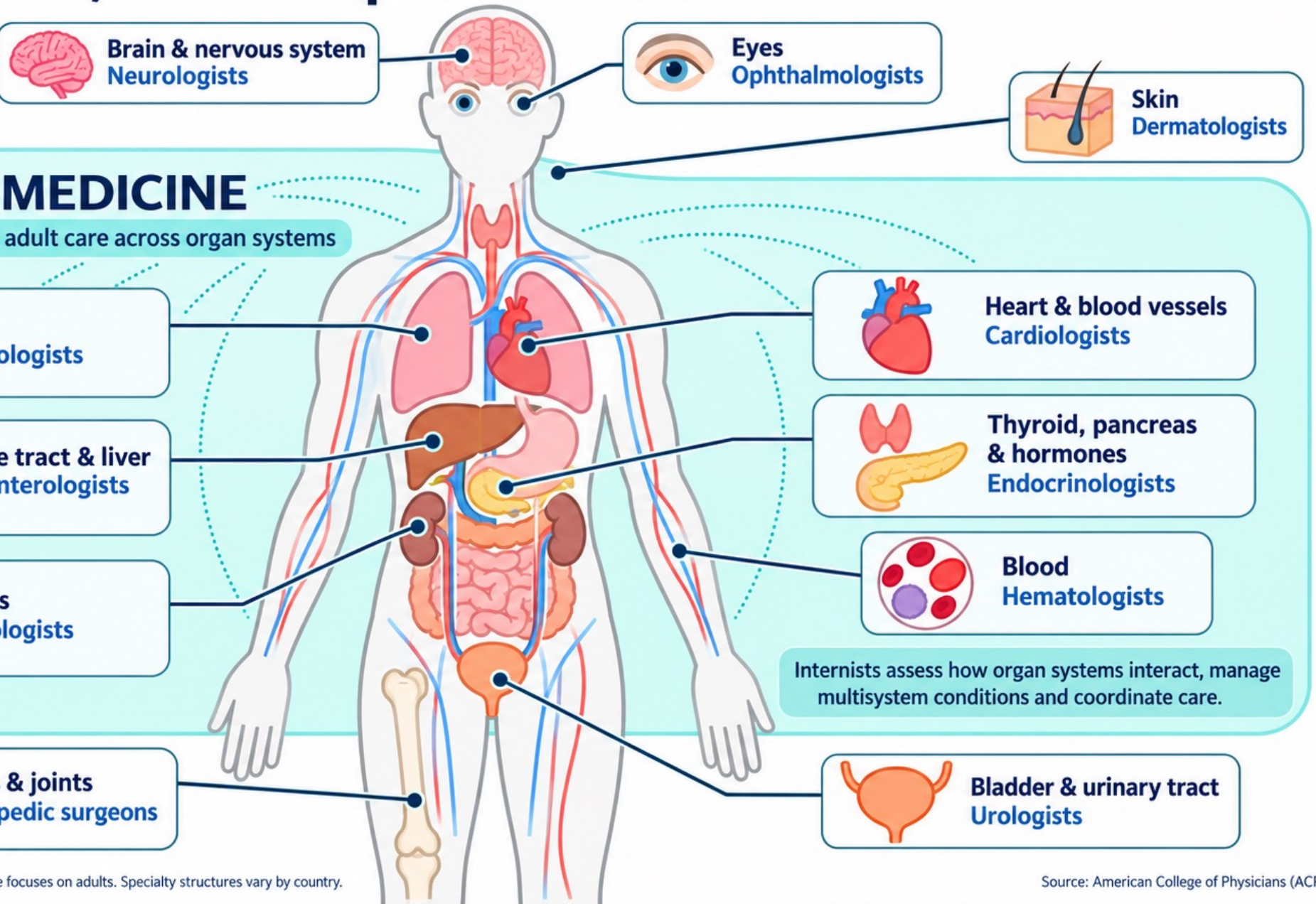
At the end of the course, you can:

- **analyze** the behavior of systems in context and **identify** problems.
- **design** practical solutions to problems and requirements, **applying** your knowledge of fundamental concepts and technologies.
- **select** suitable concepts and technologies for **developing** solutions and **justify** your design **decisions**.


I can only show you the door.

Organs, Medical Specialists & Internal Medicine

AI GENERATED



Selected examples. Specialties overlap; internal medicine focuses on adults. Specialty structures vary by country.

Source: American College of Physicians (ACP)

Lecture overview

1. Foundations
2. Distributed System Architectures
3. Communication
4. Distributed Transactions
5. Security Services
6. Resource Discovery
7. Cloud Computing
8. SOA and Business Processes
9. Processing Big Data
10. Engineering Distributed Systems

Learning Objectives of this Lecture

At the end of the lecture, you can:

- **analyze** example systems by **applying** the definition, advantages, and challenges of distributed systems to determine whether a system is a distributed system and whether it should be implemented as one.

- **define** a system model for example systems by applying the modeling levels presented in the lecture, so that the model can be used for analysis and system design.

- **analyze** example systems by applying the non-functional requirements presented in the lecture to conduct an informed system analysis or make informed design decisions for implementing the system.

What is a Distributed System (and what not)?

Relevance


T 7 3 U
livecloud.online/join/T73U
How to participate?
1 Enter your name
2 Click on join

Application Examples

We use Distributed Systems in daily live, often without even noticing it.

- Send an email
- Search information with a search engine
- Read news
- Book a ticket
- Withdraw money at an ATM
- Shop online
- Track a parcel

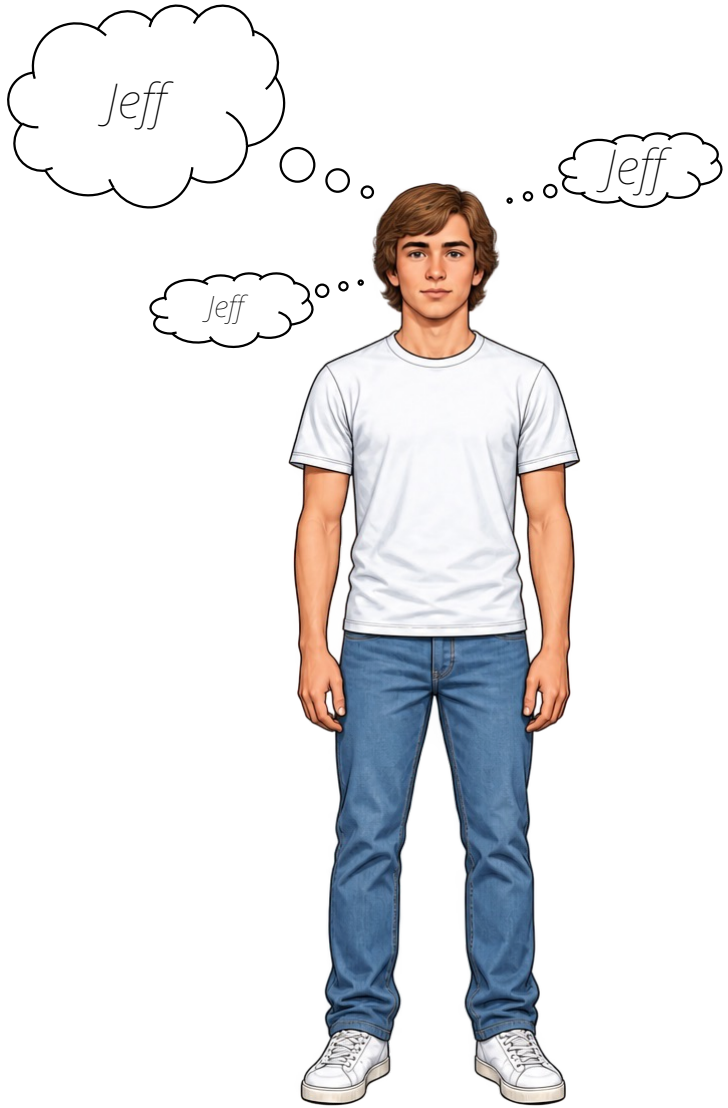
Application Examples

We use Distributed Systems in daily live, often without even noticing it.

- Send an email
- Search information with a search engine
- Read news
- Book a ticket
- Withdraw money at an ATM
- Shop online
- Track a parcel

Almost every software system today is a distributed system.

- E-Mail, Social Media Systems, Online Shopping, Banking
- Teamwork (Video conferencing, collaborative editing)
- Manufacturing systems
- Enterprise software (ERP - Enterprise Resource Planning, CRM - Customer Relationship Management, SCM - Supply Chain Management)
- Urban traffic information and control

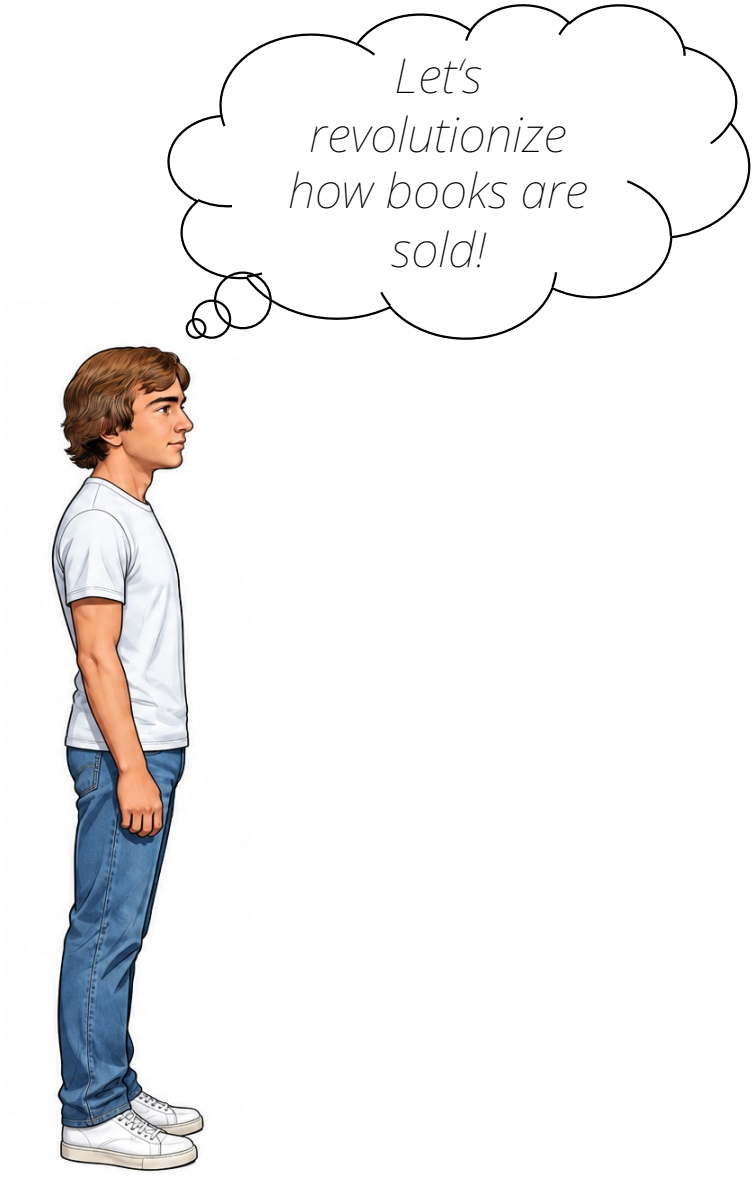

Jeff
Jeff
Jeff

Jeff likes to read books.

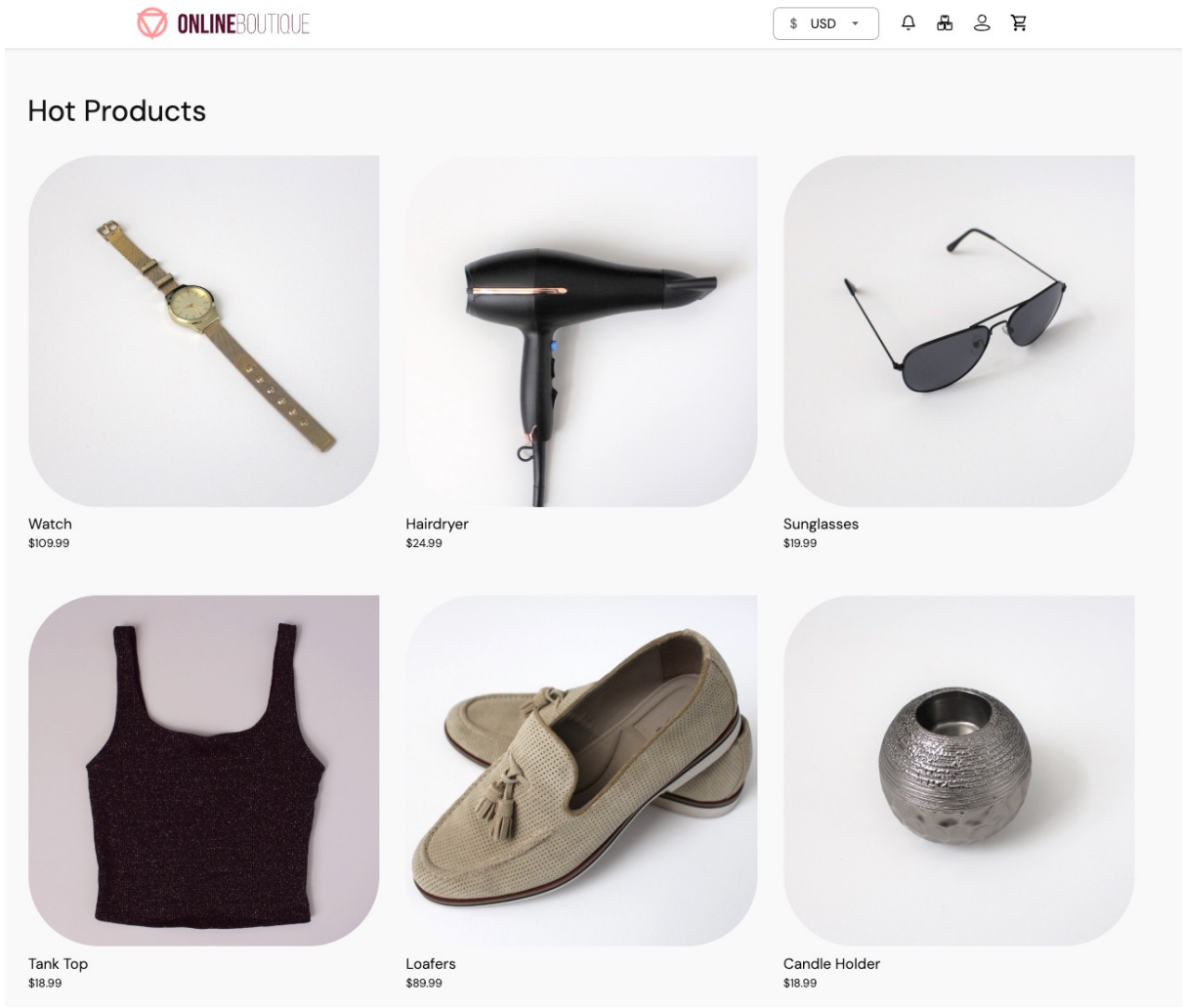



It's so annoying to go to the book store every time to search for a book!
BOOKS
Good Books Brighter Days
A KINDER WORLD THROUGH BOOKS
FICTION
MYSTERY
HISTORY
SCIENCE
HISTORY
SCIENCE
TRAVEL
BIOGRAPHY
ART
NATURE
CLASSICS


I want to do it from home!

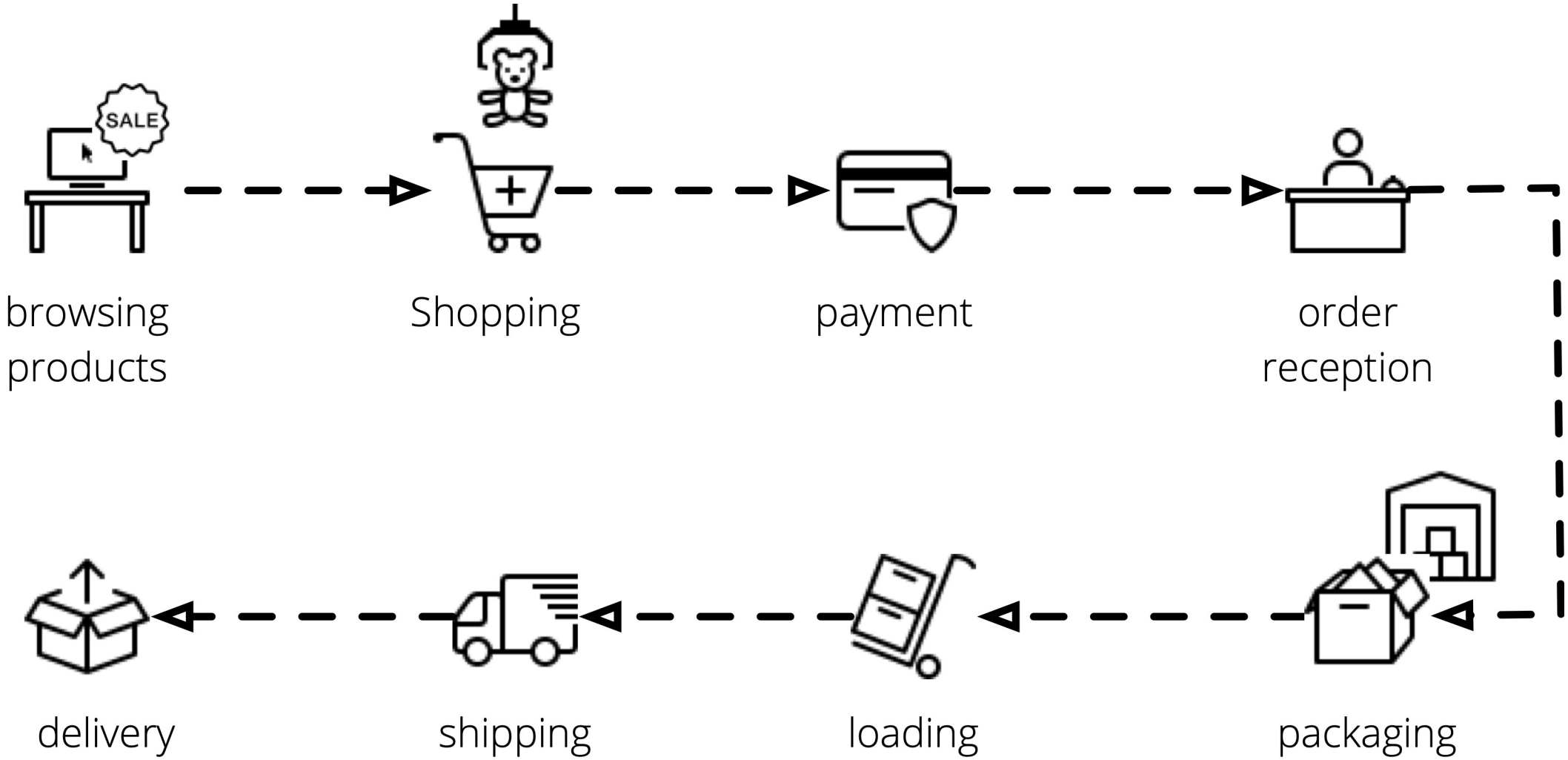

Let's revolutionize how books are sold!

Example: Onlineshop

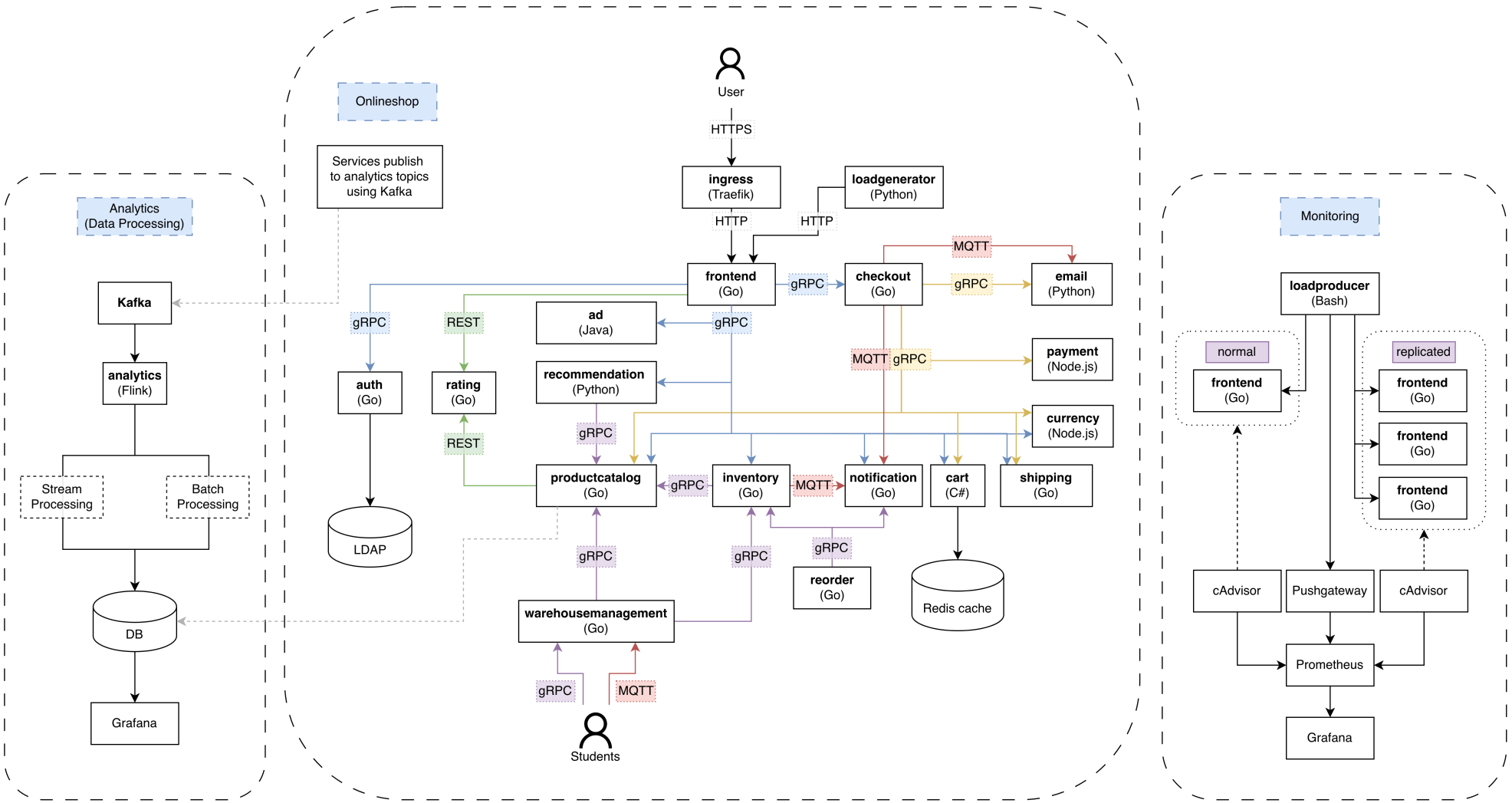


https://ds-exercise-01.netd.cs.tu-dresden.de/

Example: Online Shopping

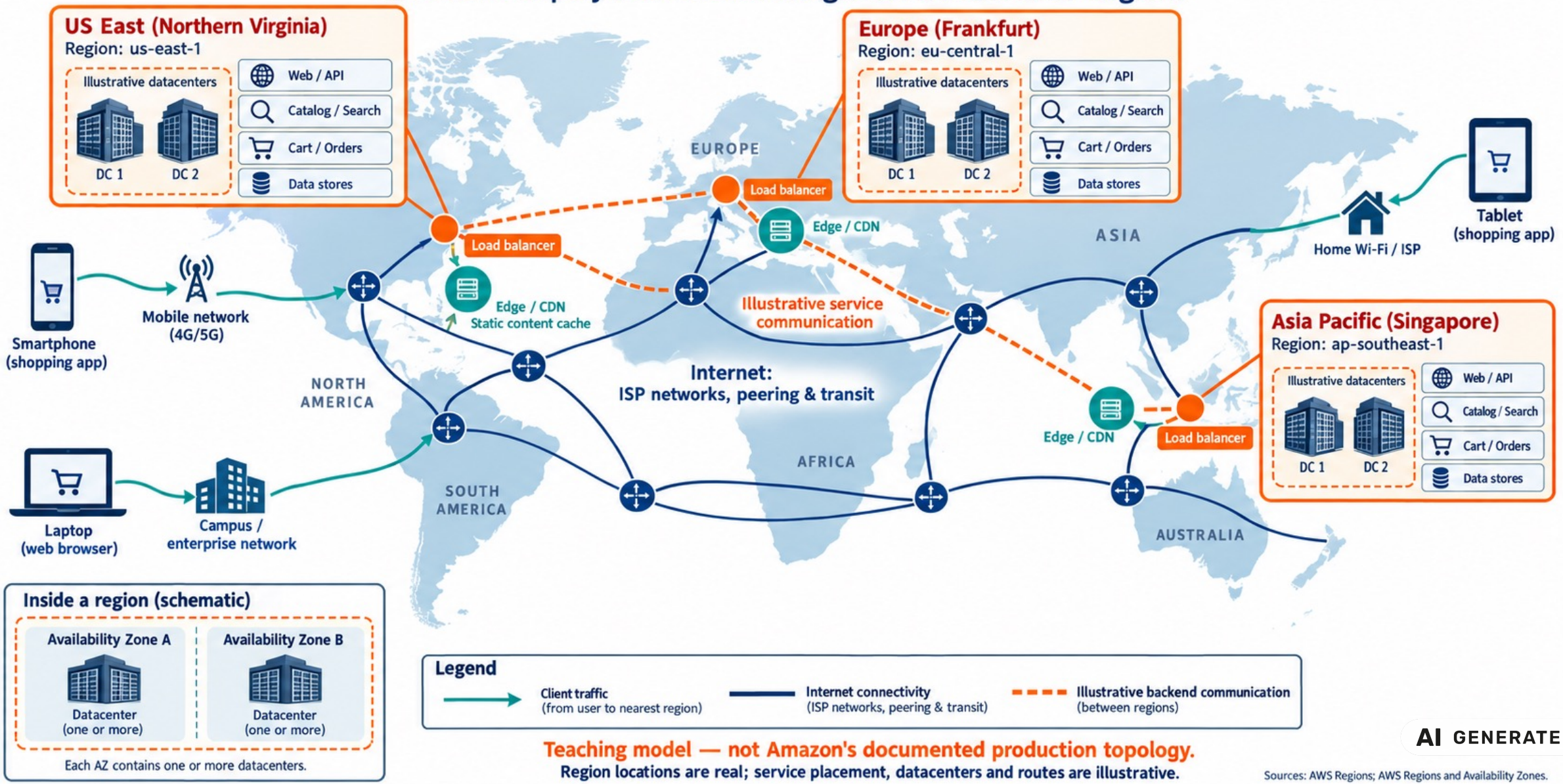


Online Shop - System Architecture



Amazon Online Shopping — A Global Distributed System

Illustrative deployment model using selected real AWS Regions



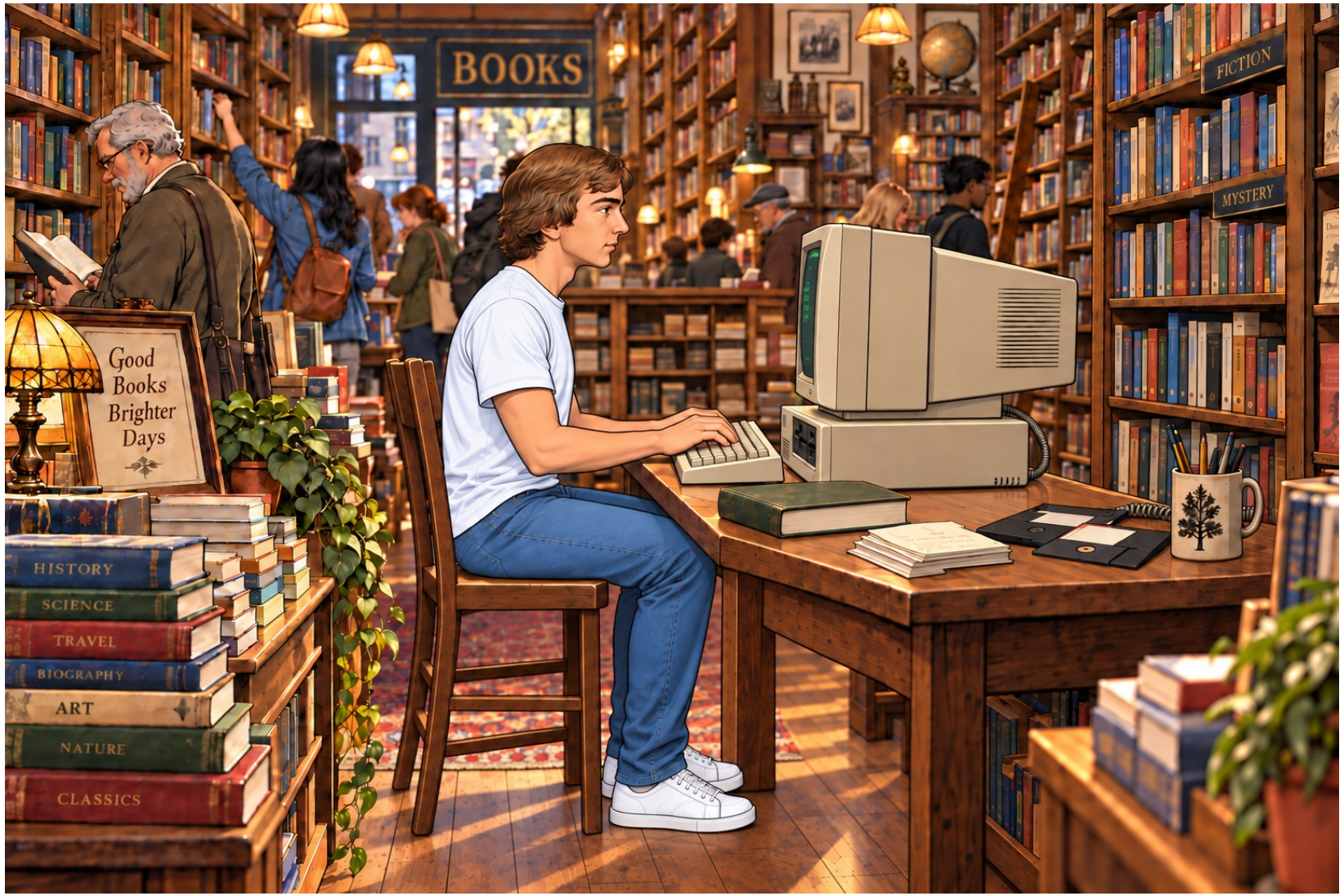
Teaching model — not Amazon's documented production topology.

Region locations are real; service placement, datacenters and routes are illustrative.

AI GENERATED

Sources: AWS Regions; AWS Regions and Availability Zones.

Let‘s do the time warp



Think-Pair-Share



Why should we use a distributed system instead of a single node system?

Resource sharing: Combine computing power, storage, and services across multiple nodes.

Higher performance: Divide work among nodes and process tasks in parallel.

Geographic proximity: Place services closer to users to reduce latency.

Availability and fault tolerance: Keep the system running even if individual nodes fail.

Scalability: Add nodes to handle more users, requests, or data.

Distribution is not for free

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

(Leslie Lamport)

Distribution is not for free

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.
(Leslie Lamport)

The Eight Fallacies of Distributed Computing (Peter Deutsch, James Gosling)

… is a list of false assumptions architects and developers might make:
(https://ably.com/blog/8-fallacies-of-distributed-computing) – drafted in 1994/1997

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. Transport cost is zero
5. The network is secure
6. The network is homogeneous
7. Topology doesn’t change
8. There is one administrator

Distribution is not for free

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.
(Leslie Lamport)

The Eight Fallacies of Distributed Computing (Peter Deutsch, James Gosling)

… is a list of false assumptions architects and developers might make:
(https://ably.com/blog/8-fallacies-of-distributed-computing) – drafted in 1994/1997

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. Transport cost is zero
5. The network is secure
6. The network is homogeneous
7. Topology doesn’t change
8. There is one administrator

Characteristics of Distributed Systems

Heterogeneity: Differences in hardware, operating systems, data formats, and protocols complicate interoperability.

Dynamism: Nodes, connections, and workloads can change during operation, requiring continuous adaptation.

Communication and Coordination Overhead: Exchanging messages and synchronizing nodes introduce latency and consume resources.

Complexity: Interactions between independent components make systems harder to design, test, debug, and operate.

Indeterminism: Concurrent execution and unpredictable message timing can produce different outcomes across otherwise identical runs.

Susceptibility to Failures: systems depend on multiple components and network connections whose availability cannot always be guaranteed.

Vulnerability: Multiple nodes and communication channels increase the attack surface and require coordinated security measures.


Share your two advantages and two challenges
Redundancy
Scalability
Synchronization
No source of truth
14:07:29
scalability
data consistency
14:07:40
challenge: may produce lower performance when searching data between multiple DB
14:13:00
consistency, communication
14:08:14
geographic performance
resource contention
14:08:19
Adv: no single point of failure
less latency
horizontal scaling
concurrency
challenges
synchronisation
14:07:26
Fault tolerance
14:07:40
concurrency , syncs
14:07:46
adcantages -more throughput, better availability
challenges - synchronisation and consistency
14:07:57
Advantage: data backup capbility, risk resistance
Challenge: cost, data sync
14:08:18
cost efficiency
scalability
14:08:32
14:07:25
Data Consistency and Synchronisation
14:07:37
Traffic handling/distribution
Better global reach
Lifetime of components providing the service
Communication overhead
14:07:45
challenges: response time, consistency, maintanence cost
14:08:08
1) scalibility
2) accessabilitetits
1) complex communication management
2) securitetis
14:08:27
partition tolerance
14:07:27
synchronization
14:07:33
challenges: High cost and complexity
14:07:40
Advantages
Perfomance can be splitted
Robustness/Backups
Challenges
Synchronization
Documentation/Knowledge about the system
14:07:51
disadvantage : complexity, data synchronization
advantage:
scalability,
resilient
1CVG
1
79
Live Wall
3 / 3
stop

Task - Indeterminism

Indeterminism is a philosophical concept that posits that not all events are causally determined by prior events. This means that some occurrences happen randomly or by chance, rather than being inevitable outcomes of a chain of cause and effect.

[Encyclopedia Britanica https://www.britannica.com/topic/indeterminism]

Task: Describe an example of indeterminisic behavior for the online shopping system.

Recent trends

Global scope
- Businesses targeting manifold regions, cultures, languages, time zones, etc.

Delivery in the cloud
- IT resources concentrated in data centers, available on-demand across the globe

Mobile and Wearable Devices
- Smartphones, tables, smart watches, wearables

Internet of things (IoT)
- devices/objects provide data about physical world

Modern web applications
- Short time-to-market and development roundtrip times

Task – Definition of Distributed System

Task: Based on your understanding of how distributed systems are structured, formulate a definition of a distributed system.

Definition

*A **distributed system** consists of independent components that run on networked computing nodes, which may be geographically dispersed. These components communicate by exchanging messages rather than accessing shared memory and cooperate to achieve a common objective.*

based on Schill, A., Springer, T.: Verteilte Systeme - Grundlagen und Basistechnologien. Springer, Berlin, 2012

Task – Defintion of Distributed Systems

Task: The definition characterizes a distributed systems by the following criteria:

It is comprised of multiple individual components. These components are able to run on spatially separated nodes. The components communicate by exchanging messages instead of using shared memory.

Which aspect from the definition is missing?

Task – Defintion of Distributed Systems

Task: Is the following system a distributed system according to the definition given in the lecture? Check every criteria you consider as fulfilled.

"A multi-processor system that runs a set of separate components for order handling that exchange data via the memory of the computer system"

Task – Is it a Distributed System

Task: Is the following system a distributed system according to the definition given in the lecture? Check every criteria you consider as fulfilled.

"A multi-processor system that runs a set of separate components for order handling that exchange data via the memory of the computer system"

System model

Characterization of system setup
Assumptions for designing systems or algorithms
Characterization of which failures are possible and which not

System model considers three major elements
Nancy A. Lynch. 1996. Distributed Algorithms. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

- Nodes
- Network
- Timing (of interactions)



System model: Nodes

Each node executes a specific piece of software.

Crash-stop:
- If a node is failing, it stops executing forever.

Crash-recovery:
- A node may crash, losing its in-memory state. It may resume executing sometime later.

Byzantine:
- A node is faulty if it deviates from the algorithm/intended behavior. Faulty nodes may do anything, including crashing or behaving maliciously.

A node that is not faulty is called “correct”.

System model: Network

Network Link:
- Bidirectional point-to-point communication between two nodes
- Unreliable due to transmission errors, human errors, intended downtimes due to maintenance

Reliable (perfect):
- A message is received if and only if it is sent. Messages may be reordered.

Fair-loss:
- Messages may be lost, duplicated, or reordered. If you keep retrying, a message is eventually delivered.

Arbitrary:
- A malicious adversary may interfere with messages (eavesdrop, modify, drop, spoof, replay)

Network partitioning: some links drop/delay all messages for an extended period of time.

System model: Timing

Timing for network links and nodes

Synchronous (highly restrictive):
- Message latency no greater than a known upper bound. Nodes execute algorithms at a known speed.

Partially synchronous:
- The system is asynchronous for some finite (but unknown) periods of time, synchronous otherwise.

Asynchronous:
- Messages may be arbitrarily delayed. Nodes can pause execution arbitrarily. No timing guarantees at all.

Task – Define a system model

Several components of the online shopping system run on multiple spatially distributed, networked nodes.

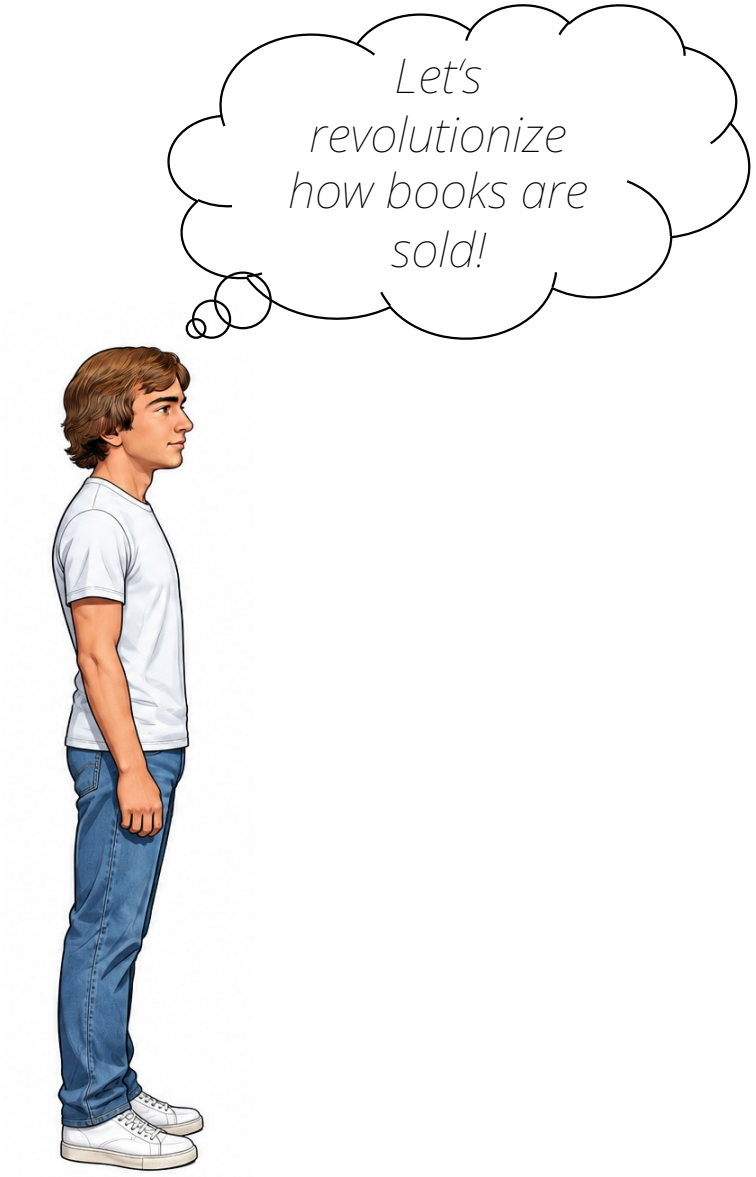
No timing guarantees can be provided for communication because the components communicate over the Internet. Messages may be lost, duplicated, or reordered.

If a node crashes, recovery is performed automatically, and the failed node is restarted after some time.

It is assumed that all system components run within a protected system infrastructure that prevents any malicious activity.

Network Link	Nodes	Timing
Reliable	Crash- stop	Synchronous
Fair-loss	Crash-recovery	Partially synchronous
Arbitrary	Byzantine	Asynchonous

REQUIREMENTS AND RELATED TERMINOLOGY


Let's revolutionize how books are sold!


I want a system
that ...

Requirements Engineering

The requirements for a system are the descriptions of what the system should do— the services that it provides and the constraints on its operation.

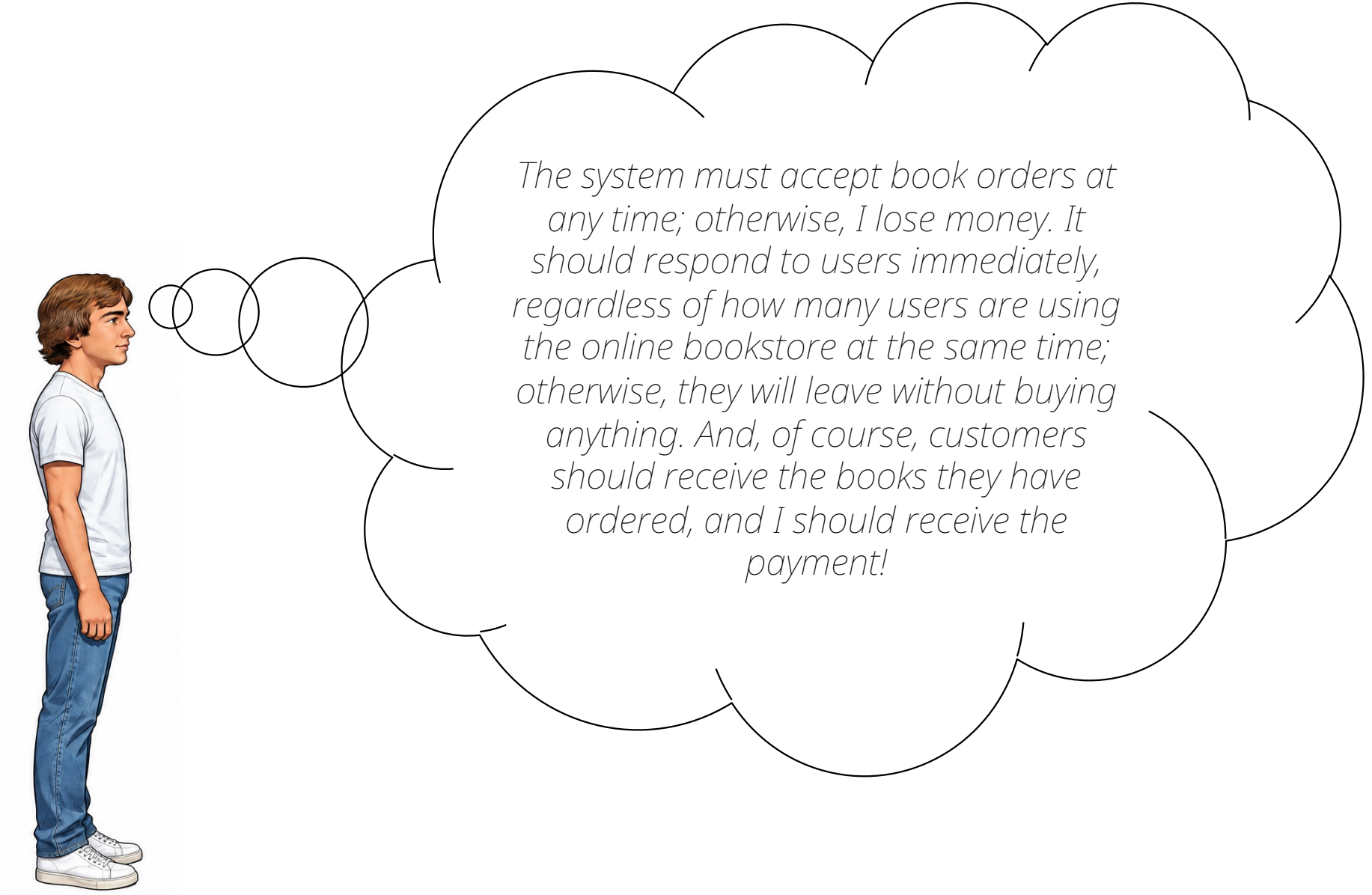
Sommerville, I., Software Engineering, Ninth Edition

Functional requirements:

- What a system should do.
- Which services or functions a system should offer.

Non-functional requirements:

- How the system should do it.
- The constraints on the services or functions offered by the system.


The system must accept book orders at any time; otherwise, I lose money. It should respond to users immediately, regardless of how many users are using the online bookstore at the same time; otherwise, they will leave without buying anything. And, of course, customers should receive the books they have ordered, and I should receive the payment!

Brainwriting Session



Task – Functional and Non-Functional Requirements

Task: Brainstorm about requirements for the online shopping system. Write down a list of requirements. Group the requirements in the list according to functional and non-functional requirements and select two from each group for sharing.

Brainwriting Session



Task – What is scalability

Task: Reflect what you know about scalability and write your defintion of scalability down in 5 sentences maximum.

Scalability: Definition

*The **scalability** of a system reflects its ability to deliver a high quality of service as demands on the system increase.*

Sommerville, I., Software Engineering, Ninth Edition

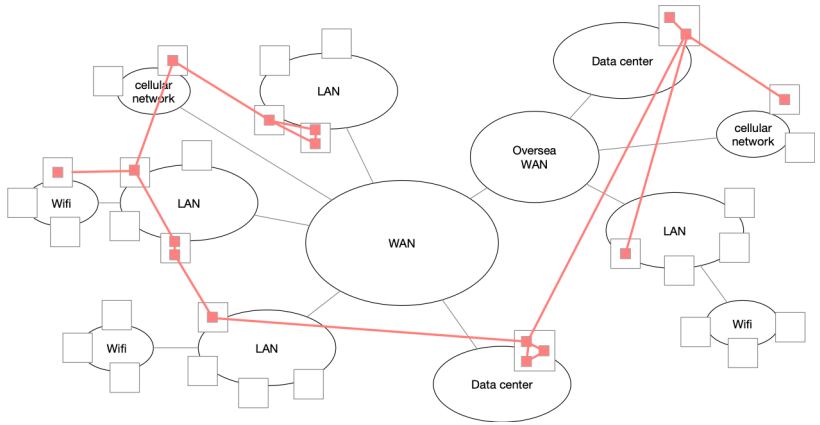
***Scalability** is a desirable attribute of a network, system, or process. The concept connotes the ability of a system to **accommodate an increasing number of elements or objects**, to **process growing or shrinking volumes of work gracefully**, and/or to be **amenable to reduction in size or to enlargement**. Usually, one thinks of the need to scale a system up so that it can cope with an increased workload; or control, manage, or monitor larger numbers of devices. In some cases, the ability to scale down a system is desirable when it is being adapted for a niche market or the setting in which the system is intended to operate is inherently smaller."*

According to Andre B. Bondi (Foundations of Software and System Performance Engineering: Process, Performance Modeling, Requirements, Testing, Scalability, and Practice: ISBN 978-0-321-83382-2, Addison-Wesley, 2015):

Scalability: System Scale

Physical deployment of a system determines the available resources

Resources allow the system to handle a certain workload with a certain performance



Online Marketing KPIs

- The *conversion rate* expresses the ratio of a website's visitors to conversions, e.g., purchases.
- Slow loading web pages reduce conversion rates and lower SEO scores
  - AlibabaCloud,2022.online,accessedNov.8th2022. url:https://www.alibabacloud.com/blog/why-is-a-websites-loading-speed-important-for-seo_598615
- People leave a web page if it does not load in approx.. 2-3 seconds
  - Nah,FionaFui Hoon."A study on tolerable waiting time:how long are web users willing to wait?."Behaviour&InformationTechnology23.3(2004):153-163.

Why changing the system scale?

Temporal effects can generate a higher workload

- Seasonal effects
- Release of new product
- Black week, Christmas

Usually, businesses start with a small number of customers and grow

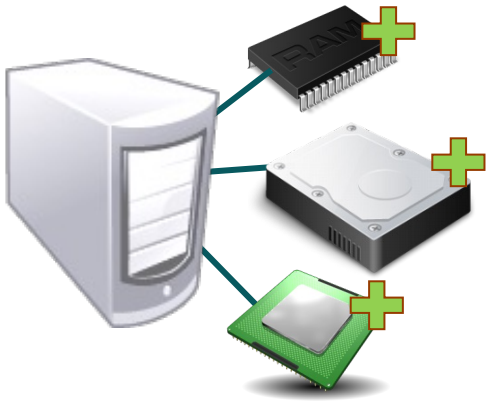
- Number of orders
- Number of customers
- Number of supported regions
- Number of products

Horizontal and Vertical Scalability: Scale Up vs. Scale Out

Scale Up

add resources to existing nodes
(vertical scaling)

+ easy to achieve
– limited to maximum hardware capacity of computing node



Scale Out

add computing nodes
(horizontal scaling)

Prerequisite: extensible software architecture

+ unlimitedly extensible
– hard to achieve: replication, partitioning and distribution of computing nodes and data has to be supported by system architecture



And what if the network breaks and not all nodes can communicate with all others. Can I still run my business?



Fault, Error and Failure

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

(Leslie Lamport)

Fault: A fault is a build-in shortcoming (defect, bug) of a system element which may but not necessarily have to cause the system component to behave incorrectly.

Error: An error occurs when a fault causes an incorrect behavior of a system component at during the execution of a system.

Failure: A failure is the observable deviation of a system from its specified correct behavior.

System components can contain **faults** → under certain circumstances **faults cause errors** during system execution → errors of system components may cause a **system failure**, if not handled or compensated

Example: Fault, Error, Failure

	Incorrect Calculation	Link error	Node error
Fault	Code computes a wrong shipping date if the order time is between 23:00 pm and 00:00 am	In rare cases of overvoltage packets in transmission can be dropped	A node from the warehouse management system is crashing and down for a certain time frame
Error	Faulty code is called during execution with order time 23:17 pm	During an overvoltage, the order processing component sends a packet to the warehouse management component that is dropped	The order processing component is sending a request to the warehouse management component while it is down and does not receive a response
Failure	An incorrect shipping date is propagated to the user and the user expects shipping on the wrong date	Due to a dropped packet the order processing component does not create an email for order confirmation	Due to the missing response the order processing component is blocked which causes the UI to freeze as well

Fault-tolerance

Fault tolerance comprises …

… techniques that ensure that faults in a system do not result in system errors or that system errors do not result in system failures.

According to Sommerville (Software Engineering, Ninth Edition, ISBN-13: 978-0-13-703515-1, Addison-Wesley, 2011

Single point of failure: an error of a system component will cause the entire system to fail.

Usually, a fault-tolerant system can tolerate faults/errors to a certain degree dependent on the effort for fault-tolerance.



Availability and Reliability

*… **availability** of a system is the probability that it will be up and running and able to deliver useful services to users at any given time*

According to Sommerville (Software Engineering, Ninth Edition, ISBN-13: 978-0-13-703515-1, Addison-Wesley, 2011

*… **reliability** of a system is the probability, over a given period of time, that the system will correctly deliver services as expected by the user.*

"Reliability includes 'correctness' (ensuring the system services are as specified), 'precision' (ensuring information is delivered at an appropriate level of detail), and 'timeliness' (ensuring that information is delivered when it is required).

According to Sommerville, Software Engineering, Ninth Edition, ISBN-13: 978-0-13-703515-1, Addison-Wesley, 2011

High Availability

Availability is usually expressed by “**the nines**“:

2 Nines	3 Nines	4 Nines	5 Nines	6 Nines
99%	99.9%	99.99%	99.999%	99.9999%

$$\text{Availability} = \frac{\text{total_seconds_in_period} - \text{seconds_downtime}}{\text{total_seconds_in_period}}$$

$$\text{Reliability} = \frac{\text{total_operations_in_period} - \text{number_of_incorrect_operations}}{\text{total_operations_in_period}}$$

High Availability

Estimated Cost of Downtime

https://www.atlassian.com/de/incident-management/kpis/cost-of-downtime

- In 2015 a 12 hour downtime of the Apple Store created cost of 25 million USD
- In March 2019 14 hours of downtime of Facebook created cost of 90 million USD
- The business model of Amazon heavily relies on availability. The cost of one hour downtime is estimated with 13,22 million USD.

High Availability starts with “four nines”

Availability in %	Downtime per year	Downtime per 30 days	Downtime per 24 hours
99,9%			
99.99%			
99.999%			

Live Questions



Task – Availability

$$\text{Availability} = \frac{\text{total_seconds_in_period} - \text{seconds_downtime}}{\text{total_seconds_in_period}}$$

Task: Fill the table.

Availability in %	Downtime per year	Downtime per 30 days	Downtime per 24 hours
99,9%			
99.99%			
99.999%			

Task - Calculate tolerable downtime

- What is the maximum downtime of a system that can be tolerated to achieve 99.99% availability within 365 days?
- What is the maximum downtime of a system that can be tolerated to achieve 99.999% availability within 30 days?
- What is the maximum downtime of a system that can be tolerated to achieve 99.9% availability within 24 hours?

High Availability

Estimated Cost of Downtime

https://www.atlassian.com/de/incident-management/kpis/cost-of-downtime

- In 2015 a 12 hour downtime of the Apple Store created cost of 25 million USD
- In March 2019 14 hours of downtime of Facebook created cost of 90 million USD
- The business model of Amazon heavily relies on availability. The cost of one hour downtime is estimated with 13,22 million USD.

High Availability starts with “four nines”

Availability in %	Downtime per year	Downtime per 30 days	Downtime per 24 hours
99,9%	8.76 hours	43.2 minutes	1.44 minutes
99.99%	**52.6 minutes**	**4.32 minutes**	**8.64 seconds**
99.999%	5.26 minutes	25.92 seconds	864.00 milliseconds

Dependability

*The **dependability** of a computer system is a property of the system that reflects its trustworthiness. Trustworthiness here essentially means the degree of confidence a user has that the system will operate as they expect, and that the system will not 'fail' in normal use.*

According to Sommerville, Software Engineering, Ninth Edition, ISBN-13: 978-0-13-703515-1, Addison-Wesley, 2011

There are four principal dimensions to dependability:

- Availability
- Reliability

- *Informally, the **safety** of a system is a judgment of how likely it is that the system will cause damage to people or its environment.*

- Informally, the ***security*** of a system is a judgment of how likely it is that the system can resist accidental or deliberate intrusions.

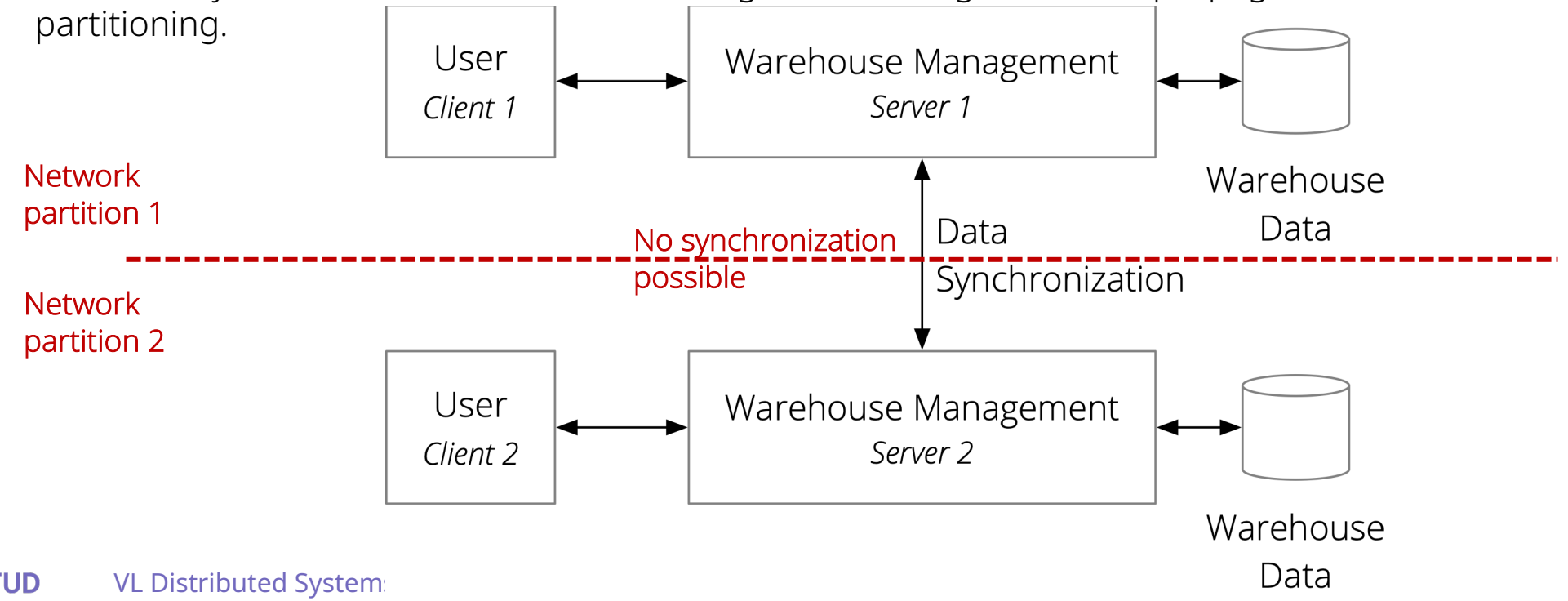
CAP Theorem

Explains trade-off between Consistency, Availability and Partioning tolerance

Brewer, E.: CAP twelve years later: How the ”rules” have changed. IEEE Computer, 45(2), 23–29 (February 2012)

If a system is partitioned, either consistency or availability can be achieved but not both

- If availability is maintained, data can be changed, but changes can't be propagated to all nodes due to the partitioning.



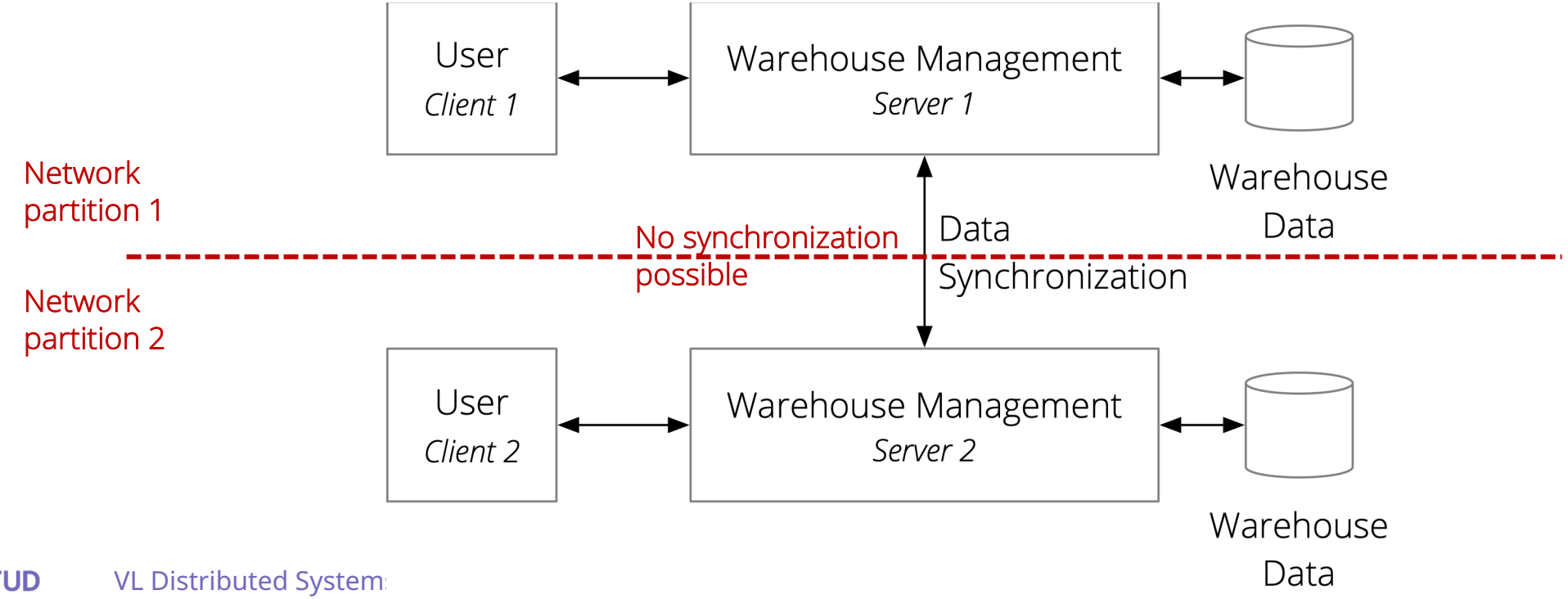
CAP Theorem

Explains trade-off between Consistency, Availability and Partioning tolerance

Brewer, E.: CAP twelve years later: How the ”rules” have changed. IEEE Computer, 45(2), 23–29 (February 2012)

If a system is partitioned, either consistency or availability can be achieved but not both

- If consistency is maintained, data can't be changed which harms the availability.



Example for “relaxed” CAP

The CAP theorem is often interpreted as too strict since it is usually not either 100% Availability or 100% Consistency, but some compromise in between.

Example: Online Shop that maintains distributed warehouses and the number of articles available should be consistent to ensure correct shipping info to users

Problem: If the system is partitioned, with the strict interpretation, either no orders are possible to maintain consistency of article numbers, or running orders could cause shortages of particular articles due to inconsistent article numbers

Possible strategies:

	Article number	Order volume	Shipping date	Compensation
A	Show latest article number	**Restrict to small orders**	Use short estimated shipping date	No compensation
B	Show latest article number	Do not restrict order volume	**Use conservative shipping date**	No compensation
C	**Show availability only**	Do not restrict order volume	Use short estimated shipping date	**Compensate late shipping**

Summary

*A **distributed system** consists of independent components that run on networked computing nodes, which may be geographically dispersed. These components communicate by exchanging messages rather than accessing shared memory and cooperate to achieve a common objective.*

based on Schill, A., Springer, T.: Verteilte Systeme - Grundlagen und Basistechnologien. Springer, Berlin, 2012

System characteristics (compared to a single node system)

- Overhead, complexity, heterogeneity, dynamism, reliability, vulnerability, indeterminism

System model

- Network links, nodes, timing

Non-functional requirements

- Scalability, availability, reliability, dependability, fault tolerance, CAP Theorem

References

Schill, A., Springer, T.: Verteilte Systeme - Grundlagen und Basistechnologien. Springer, Berlin, 2012

van Steen, M., Tanenbaum, A.S.: Distributed Systems – Principles and Paradigms, Pearson, 2017

Sommerville, I. Software Engineering, Ninth Edition, ISBN-13: 978-0-13-703515-1, Addison-Wesley, 2011

Andre B. Bondi, Foundations of Software and System Performance Engineering: Process, Performance Modeling, Requirements, Testing, Scalability, and Practice: ISBN 978-0-321-83382-2, Addison-Wesley, 2015

Nancy A. Lynch. 1996. Distributed Algorithms. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

Brewer, E.: CAP twelve years later: How the ”rules” have changed. IEEE Computer, 45(2), 23–29 (February 2012)